PROJECT DESIGN • 2026

SynapseMind

突触智学 · 单题诊断引导

输入一道题 + 你的答案，找到你卡住的那一步

不是讲这道题，是找到你卡住的那一步。

把“我不会”从一句情绪，变成一个可命名、可验证、可训练的卡点。

学生发来一道题 + 自己的答案；系统用 2–6 轮追问定位具体卡点，再给 3 分钟微干预与一道迁移题。

高中数学 • 函数与导数 | 100 题 • 30 题深标 | 3 人 • 8 周 | 4 个部署单元 → 7 个服务

项目设计方案 v0.2 • 2026 年 10 月

本文为设计方案，不是已上线产品。所有指标均为设计目标或估算；缺口见第 9.8 节。

目录

0. 一页纸 6
0.1 用户带什么来 6
0.2 系统做什么 6
0.3 一个真实片段 6
0.4 我们不是什么 7
0.5 一句话价值 7
0.6 核心指标 8
1. 需求：用户带来的不是“题”，是“卡住” 9
1.1 三个真实场景 9
1.2 现有产品断在哪 9
1.3 项目的原子单位 10
1.4 我们的承诺 10
1.5 边界 10
1.6 为什么现在能做 11
2. 核心体验：一次 8 分钟的单题诊断 12
2.1 七步闭环 12
2.2 完整示例：答案对，过程漏了条件 13
2.3 分支：用户说“完全不会” 15
2.4 分支：只给了一个最终答案 16
2.5 分支：答案对，但心里没底 16
2.6 对话界面的四条硬规则 17
2.7 为什么用户会信任 17
3. 科学原理如何变成对话规则 18
3.1 六条原理 → 六条对话规则 18
3.2 七个能力维度：只做“探针镜头”，不做分数 18

3.3 认知负荷：把“大问题”切成“能回答的小问题” 19
3.4 预测误差：让“哦”发生在对话里 19
3.5 提取练习与自我解释：不问“懂了吗” 19
3.6 巩固与迁移：诊断不是终点 19
3.7 神经可塑性的正确用法 20
3.8 伦理红线 20
3.9 一句话总结 20
4. 诊断引擎：从“答案”到“卡点” 21
4.1 题目不是一段文本，是一张解法图 21
4.2 学生作答不是“对/错”，是一条可对齐的步骤链 22
4.3 卡点分类：L0 – L6 23
4.4 假设空间：一次只比较 2 – 3 个可能 23
4.5 探针选择：问“最有区分度”的那个 24
4.6 置信度更新与停止条件 24
4.7 诊断卡：不是“报告”，是“下一动作” 25
4.8 失败与降级 25
4.9 一个最小可用的解法图示例 25
5. Prompt 与 Agent：让“追问”可测试 27
5.1 四个运行时角色 27
5.2 状态机：不是“多 Agent 聊天” 27
5.3 系统提示词（核心版） 28
5.4 输出 Schema 28
5.5 好问题 / 坏问题 28
5.6 答案泄漏的四道闸 29
5.7 MCP：把能力暴露给 IDE，而不是再写一套 API 29
5.8 Agent Skills：把“怎么追问”变成可发布资产 30
5.9 模型路由与成本 30
5.10 失败时的兜底 30

6. RAG 与内容：让追问有据可查 31

6.1 知识库不是“教材切片”，是五类卡片 31

6.2 文档 ETL 31

6.3 切分策略 31

6.4 检索：混合召回 + 重排 32

6.5 pgvector 配置 32

6.6 内容生产：100 题，30 题深标 33

6.7 提示卡：0–3 级，不许跳级 33

6.8 引用与可追溯 33

6.9 内容侧的 SEO/GEO 34

7. 技术架构：为“一次对话”设计，不为“大平台”设计 35

7.1 四个部署单元，七个服务边界 35

7.2 核心表 36

7.3 一次诊断的调用链 37

7.4 缓存与并发 37

7.5 可观测性 38

7.6 部署 39

7.7 安全与合规 39

7.8 为什么不是“一上来微服务” 39

8. skill.txt 逐条映射：每个能力落在哪个文件、哪次演示 40

8.1 开发技术 1–7 40

8.2 AI 应用开发 1–4 41

8.3 覆盖度自检 41

8.4 面试时怎么讲 42

8.5 90 秒陈述 42

8.6 简历写法（标明设计/目标） 42

9. 可行性与 MVP：3 个人 8 周，先证明“说得准” 44

9.1 MVP 范围 44

9.2 八周计划 44
9.3 评测：怎么证明“说得准” 45
9.4 成本 45
9.5 风险登记 46
9.6 Kill criteria 46
9.7 为什么是 8 周，不是 10 周 47
9.8 诚实清单 47
10. 3 分钟演示与高频追问 48
10.1 演示脚本 48
10.2 高频追问 48
10.3 验收清单 50
11. 设计过程：5 个“无共享记忆”的子 Agent 51
11.1 为什么要无共享记忆 51
11.2 五个角色与提示词 51
11.3 主 Agent 裁决的 6 个冲突 52
11.4 红队改变了什么 52
11.5 这套方法怎么复用到别的项目 52
11.6 与运行时多 Agent 的区别 52
附录 54
A. 诊断卡 JSON 54
B. 追问生成 Prompt（节选） 54
C. 解法图 JSON（示例） 54
D. 术语表 55
E. 参考方向 55
F. 版本 56

0. 一页纸

> 不是讲这道题，是找到你卡住的那一步。

0.1 用户带什么来

三种入口，同一个动作：

入口	用户提供	系统第一句话
我不会	题目文本/拍照，最多加一句“我读到哪就懵了”	“先不做整题。只看这一步，你能判断吗？”
我做错了	题目 + 自己的答案/过程 + 正确答案（可选）	“先不看正确答案。你写到这一步时，心里是怎么想的？”
答案对但没底	题目 + 答案 + “我是猜的/我不确定”	“答案先放一边。你能用自己的话解释这一步为什么成立吗？”

0.2 系统做什么

不是直接给解答，而是用 **2-6 轮追问**，把“我不会”拆到一个可命名的卡点：

1. **拆题**：把题目拆成解法图（读题、概念、条件转译、策略、执行、校验）。
2. **对齐**：把用户的答案/步骤贴到解法图上，找第一处偏离。
3. **追问**：一次只问一个可回答的问题，用具体例子、反例、二选一，逼出真实理解。
4. **定位**：输出一张“卡点卡”——卡在哪一层、证据是什么、排除了什么、置信度多少。
5. **微干预**：3 分钟讲清一个点，配 1 道同构题 + 1 道迁移题。
6. **验证**：24 小时/7 天后用一道新题复测，确认不是“听懂了”。

0.3 一个真实片段

题目：$f(x)=\ln x - a x$，$a\in R$，若 $f(x)\le 0$ 对任意 $x>0$ 恒成立，求 a 的取值范围。

学生答案：$f'(x)=1/x-a$，令 $f'(x)=0$ 得 $x=1/a$，最大值 $\ln(1/a)-1\le 0$，所以 $a\ge 1/e$。**答案对，过程漏了 `a≤0` 的讨论。**

系统没有说“你漏了 a≤0”，而是问：

> “先不急着看答案。你说‘令 f’(x)=0 得 x=1/a’，这一步用了除法。请只回答：当 $a=0$ 时，$f'(x)$ 是什么？在 $x>0$ 上有零点吗？”

学生：“$1/x$，没有零点。”

> “那 a=0 时，f(x)=ln x 在 x>0 上会不会超过 0？”

学生：“会，x>1 就大于 0。”

> “所以 a=0 不符合。再看 a=-1：f'(x)=1/x+1，有零点吗？f(x) 在 x>0 上单调怎样？”

学生：“没有零点，一直增；x 很大时 ln x + x 也很大，肯定超过 0。”

> “很好。现在回到你的第一步：‘令 f'(x)=0 得 x=1/a’，它要求 a 满足什么？”

学生：“a>0。a<0 时 1/a 是负的，不在 x>0；a=0 也不行。所以只有 a>0 时这个驻点才在定义域里。”

> “对。你原来的解答没有检查这个前提。那你觉得，这次卡住的是求导、解不等式，还是‘用驻点前先确认参数范围’？”

学生：“最后一个。”

卡点卡：

项	内容
卡点	L2 条件转译 / 参数分类讨论：使用驻点前，未确认 a>0 使 x=1/a 存在且落在 x>0
证据	在 a=0、a=-1 时仍沿用 x=1/a；没有先判断驻点是否存在、是否为最大值
排除	求导正确；不等式求解正确；最终答案正确
置信度	0.86
下一练	用“存在性三步检查”重写开头；再做一道同构题（含参数分类）
验证	24 小时后，新题中能否先写“驻点存在的条件”，再求导

0.4 我们不是什么

不是	为什么
题库/搜题	搜题给答案，我们给“你卡在哪”；答案在诊断完成前是锁住的
AI 家教	家教讲整道题，我们只追问一个点，讲 3 分钟就停
智商测试	不输出 IQ、百分位、等级；七维能力只在后台做“探针镜头”
完整课程	MVP 只做高中数学“函数与导数”，不承诺覆盖所有学科
提分承诺	只承诺“更准确地找到下一个该练的点”，不承诺分数

0.5 一句话价值

把“我不会”从一句情绪，变成一个可命名、可验证、可训练的卡点。

0.6 核心指标

指标	MVP 目标	说明
诊断与双教师一致率	≥75%	同一段对话，两位老师独立标注卡点，系统取一致
用户“说中了”	≥70%	诊断卡下方 1–5 分，“这个判断像不像你”
平均追问轮数	4–6 轮	超过 6 轮必须给阶段性结论
首条有效追问延迟	<8 秒	含 OCR/解析/检索
答案泄漏率	0	学习模式下，输出中不出现最终答案/完整解法
24 小时迁移	≥60%	新题中不再犯同一卡点
单次成本	≤¥0.8	含 LLM、OCR、向量检索

1. 需求：用户带来的不是“题”，是“卡住”

1.1 三个真实场景

场景 A：完全不会。

一个高一学生拍来一道题：

> 已知函数 $f(x)=x^3-3x+1$，讨论方程 $f(x)=0$ 的实根个数。

他只打了一句：“完全不会。”

传统产品的反应是：识别题目 → 给完整解答。学生看完，“哦”一声，下次遇到同型题还是不会。因为他不会的不是“这道题的答案”，而是“怎么想到用零点定理 + 单调性 + 端点值”。

场景 B：做错了，但不知道错在哪。

一个高二学生发来题目和自己的答案：

> 题目：$f(x)=\ln x - a x$，$a\in R$，若 $f(x)\le 0$ 对任意 $x>0$ 恒成立，求 a 的取值范围。 学生：$f'(x)=1/x-a$，令 $f'(x)=0$ 得 $x=1/a$，最大值 $\ln(1/a)-1\le 0$，所以 $a\ge 1/e$。

答案是对的，过程漏了 $a\le 0$。如果系统只说“你漏了 $a\leqslant 0$”，学生记住的是“这题要分类”，下次换一个参数，仍然不知道“什么时候需要先检查驻点存在”。

场景 C：答案对，但心里没底。

学生说：“我选 C，但我是排除法猜的。”

这时最危险：分数掩盖了理解的空洞。系统要问的不是“为什么选 C”，而是“你能构造一个反例，说明 B 为什么不对吗？”

1.2 现有产品断在哪

产品形态	它解决了什么	它没解决什么
搜题/题库	快速拿到答案	不知道“我为什么没想到”
AI 家教	把解答讲清楚	讲的是“题”的逻辑，不是“你”的断点
错题本	把错题存下来	只记录“错了”，不记录“错在哪个认知动作”
知识点标签	告诉我“函数单调性薄弱”	太粗：同一个标签下，可能是读题、概念、策略、计算、校验五种病
自适应刷题	根据正确率推题	正确率是结果，不是过程；蒙对和真会无法区分

空白点：从“这道题我错了”到“我具体缺哪个认知动作”之间，缺一层**可解释的诊断**。

1.3 项目的原子单位

不是“学生”，也不是“题目”，而是：

> **一次作答 = 一个学生 × 一道题 × 一条过程。**

同一道题，三个学生可能卡在三处：

- 学生 A：读不懂“对任意 x>0 恒成立” → L0 语义。
- 学生 B：不知道 `f'(x)=0` 只是必要条件，还要判断极大/极小 → L1 概念。
- 学生 C：知道要分类，但一分类就漏 `a=0` → L2 条件转译 + L4 执行。

如果系统只给“知识点：导数应用”，这三个人会拿到同一份练习。这就是无效刷题的根源。

1.4 我们的承诺

8 分钟内，给你一个可命名的卡点，而不是一整道题的解答。

具体承诺：

1. 第一轮追问在 8 秒内出现；
2. 最多 6 轮，必须给出阶段性结论；
3. 结论必须带证据（引用你刚才的原话/步骤）；
4. 必须给出“排除了什么”，防止把多因一果简单化；
5. 必须给一个 3 分钟能做完的下一步；
6. 学习模式下，不直接给最终答案。

1.5 边界

维度	MVP 边界	原因
学科	高中数学“函数与导数”	解法图相对可控；SymPy 可验证
题型	求参数范围、单调性、极值、零点个数、恒成立	覆盖高频且可分步
输入	文本/拍照；答案可手写/文本	OCR 是最大工程风险，先做 Mathpix 类兜底
输出	卡点卡 + 3 分钟微干预 + 1 道迁移题	不承诺完整课程
用户	高一至高三	不含小学/大学
不做	医学诊断、智商分数、排名、提分承诺	伦理与合规红线

1.6 为什么现在能做

三件事同时成熟：

1. **多模态输入**：拍照 + LaTeX OCR 已经可用，学生不需要敲公式。
2. **结构化输出**：LLM 能稳定输出 JSON，把“追问”变成可测试的状态机，而不是聊天。
3. **小模型成本下降**：一次诊断 4–6 轮，每轮 1–2k token，成本可以压到 ¥0.8 以内。

但真正的壁垒不是模型，是：

> **把“老师会怎么追问”拆成可复用的探针库，并且让每一道题都有可对齐的解法图。**

这部分是脏活，也是本项目最值得讲的地方。

2. 核心体验：一次 8 分钟的单题诊断

2.1 七步闭环

步骤	时间	系统动作	用户看到
1. 交题	0:00 – 0:30	OCR/文本解析，识别题目、答案、卡住位置	“已收到。先确认一下题目识别对不对。”
2. 拆题	0:30 – 1:30	生成解法图：节点、依赖、每步的能力需求	一张极简“这题分几步”的图，不展开答案
3. 对齐	1:30 – 2:30	把用户步骤贴到解法图，找第一处偏离	“你写到第 2 步时，用了一个前提，我们来看看它。”
4. 追问	2:30 – 6:00	2 – 6 轮探针，一次一问，按信息增益选下一个问题	一个具体问题 + 可点的选项/输入框
5. 定位	6:00 – 6:30	更新假设后验，生成卡点卡	“你卡在 L2，不是不会求导。”
6. 微干预	6:30 – 7:30	3 分钟讲解 + 1 道同构题	一个概念卡 + 一道小题
7. 验证	7:30 – 8:00	预约 24h/7d 迁移题；记录证据	“明天用一道新题验证，不用同一道。”

输入字段：

字段	必填	说明
`problem`	是	题目文本或拍照
`student_answer`	否	最终答案；没有就填“不会”
`student_work`	否	过程/步骤；越完整，诊断越准
`stuck_at`	否	学生自述“我读到哪/写到哪就懵了”
`confidence`	否	这题有几成把握（1 – 5）
`correct_answer`	否	正确答案；只用于校验，学习模式不展示



图 2-1 单题诊断闭环：交题 → 拆题 → 对齐 → 追问 → 定位 → 微干预 → 迁移验证。

2.2 完整示例：答案对，过程漏了条件

题目

> 已知 f(x)=ln x - a x，a∈R。若 f(x)≤0 对任意 x>0 恒成立，求 a 的取值范围。

学生提交

> f'(x)=1/x-a。令 f'(x)=0，得 x=1/a。最大值 f(1/a)=ln(1/a)-1。令 ln(1/a)-1≤0，得 a≥1/e。所以 a∈[1/e,+∞)。

第 1 轮：不纠正，先要一个可判断的具体例子

> 系统：先不急着看答案。你说“令 f'(x)=0 得 x=1/a”，这一步用了除法。请只回答：当 a=0 时，f'(x) 是什么？在 x>0 上有零点吗？
>
> 学生：1/x，没有零点。
>
> 系统：那 a=0 时，f(x)=ln x 在 x>0 上会不会超过 0？
>
> 学生：会，x>1 就大于 0。

第 2 轮：再给一个反例，确认不是“只忘了 a=0”

> 系统：所以 a=0 不符合。再看 a=-1：f'(x)=1/x+1，有零点吗？f(x) 在 x>0 上单调怎样？

学生：没有零点，一直增；x 很大时 `ln x + x` 也很大，肯定超过 0。

系统：很好。现在回到你的第一步：“令 `f'(x)=0` 得 `x=1/a`”，它要求 `a` 满足什么？

学生：`a>0`。a<0 时 1/a 是负的，不在 x>0；a=0 也不行。所以只有 a>0 时这个驻点才在定义域里。

第 3 轮：把“知识点”翻译成“认知动作”

系统：对。你原来的解答没有检查这个前提。那你觉得，这次卡住的是求导、解不等式，还是“用驻点前先确认参数范围”？

学生：最后一个。

系统：如果让你给这一步起个名字，你会叫它什么？

学生：先看驻点存不存在。

第 4 轮：确认诊断，给用户否决权

系统：我的判断是：你卡在“条件转译/参数分类”，不是求导，也不是解不等式。证据是：你在 a=0、a=-1 时仍沿用了 x=1/a。这个判断像你吗？（1 - 5）

学生：4。

诊断卡

```
┌ 卡点卡 ──────────────────────────────────────────┐
│ 卡点   L2 条件转译 / 参数分类讨论                 │
│ 具体   使用驻点前，未确认 a>0 使 x=1/a 存在且落在  │
│        x>0；也未排除 a≤0 时函数无最大值           │
│ 证据   在 a=0、a=-1 时仍沿用 x=1/a                │
│ 排除   求导正确；不等式求解正确；最终答案正确      │
│ 置信   0.86（高）                                 │
│ 不是   “粗心”；不是“不会求导”；不是“不会解不等式”  │
│ 下一练 用“存在性三步检查”重写开头：               │
│        ① 参数是否使驻点存在？                     │
│        ② 驻点是否在定义域？                       │
│        ③ 驻点是极大还是极小？                     │
│ 验证   24h 后一道新题：能否先写存在性条件，再求导  │
└──────────────────────────────────────────────────┘
```

3 分钟微干预

你不是不会求导，而是把“驻点”当成了一个不用检查就能用的东西。 对 `f(x)=ln x - a x`，`f'(x)=1/x - a`。只有当 `a>0` 时，`1/x=a` 才有正解 `x=1/a`。当 `a≤0` 时，`f'(x)>0` 在 `x>0` 上恒成立，函数单调递增；x 趋于无穷时 `f(x)` 也趋于无穷，不可能恒 ≤0。 所以正确的开头不是“令 f’(x)=0”，而是： 1. 先看 `a≤0`：单调递增，排除； 2. 再看 `a>0`：驻点存在，且 `f''(x)=-1/x²<0`，是最大值； 3. 最后解 `ln(1/a)-1≤0`。 记住一句话：**“先问驻点存不存在，再问它是不是最大值。”**

同构题

> $f(x)=x - a \ln x$，$a\in R$，若 $f(x)\geq 0$ 对任意 $x>0$ 恒成立，求 a 的范围。先只写“存在性检查”的三行，不用算完。

2.3 分支：用户说“完全不会”

“完全不会”不是没有信息，而是信息在更前面。系统不能直接讲解答，而要找到“最后一个会的点”。

题目： $f(x)=x^3-3x+1$，讨论 $f(x)=0$ 的实根个数。

> 系统：没关系，先不做整题。只看这三个值：$f(-1)$、$f(0)$、$f(1)$ 分别是多少？
>
> 学生：$f(-1)=3$，$f(0)=1$，$f(1)=-1$。
>
> 系统：好。$f(0)=1$，$f(1)=-1$。中间有没有至少一个根？为什么？
>
> 学生：有，连续，零点定理。
>
> 系统：那 $f(-1)=3$，$f(0)=1$，能说明有根吗？
>
> 学生：不能，都是正的。
>
> 系统：所以你已经会用零点定理了。现在卡的是“怎么选区间”，还是“怎么知道每个区间里只有一个根”？
>
> 学生：第二个。
>
> 系统：那要判断“只有一个”，需要知道函数的什么性质？
>
> 学生：单调性。
>
> 系统：对。你看，你不是“完全不会”，你是“会零点定理，但还没把单调性和根的唯一性接起来”。我们只练这一座桥。

诊断卡：

项	内容
卡点	L3 策略：不会把“根的个数”拆成“零点定理 + 单调性 + 端点趋势”
已知	会算函数值；会用零点定理判断存在性
缺口	不会用导数判断单调区间；不会把“唯一”转化为“单调”
下一练	先只做“求单调区间”，不做根的个数；再加零点定理
置信度	0.78

2.4 分支：只给了一个最终答案

学生：答案 `a≥1/e`。（没有过程）

系统不能假装能对齐步骤。第一步不是判断对错，而是**补过程**：

> 系统：先不判断对错。你能补一步吗：你第一步是求导、画图、还是分类？选一个就行。
>
> 学生：求导。
>
> 系统：好。`f'(x)` 等于什么？只写这一行。
>
> 学生：`1/x - a`。
>
> 系统：令它等于 0，你解出什么？
>
> 学生：`x=1/a`。

到这里，系统已经拿到两个关键步骤，可以开始追问“a=0 时怎么办”。如果学生连一步都不愿补，系统降级为“答案反推”：

> “如果 a=0，你的答案 a≥1/e 还成立吗？代进去看看。”

诊断卡会标注：

> 过程证据不足，置信度上限 0.65；当前判断是“可能漏参数分类”，建议补一步过程后再确认。

原则：没有过程证据，就不假装高置信。诊断可以“暂时不确定”，但不能编。

2.5 分支：答案对，但心里没底

学生：这题选 C，但我是排除法猜的。

> 系统：先不解释为什么 C 对。你能构造一个反例，说明 B 为什么不对吗？
>
> 学生：……想不到。
>
> 系统：那我给你一个具体值，你代进去试试：`x=0` 时，B 选项的式子等于多少？
>
> 学生：等于 0，但题目说 x>0，所以不能取 0。
>
> 系统：很好，你已经找到 B 的一个边界问题。那 C 呢？你能举一个满足条件、且让 C 成立的例子吗？
>
> 学生：可以，x=1 时……
>
> 系统：所以你不是“完全不懂”，你是“会用排除法，但还没有用正例确认”。这次卡在 L5 校验，不是 L1 概念。

要点：答案正确不等于理解正确。系统要区分“真会”“蒙对”“排除法”“背套路”，否则会把运气记成能力。

2.6 对话界面的四条硬规则

1. **一次只问一个问题。** 不问“你觉得哪里不会？还有呢？”——这是把认知负担丢回给用户。
2. **问题必须可回答。** 优先用“二选一 / 具体数值 / 下一步做什么”，少用“为什么”。
3. **不在一开始给答案。** 前 3 轮不出现最终结果、完整解法、关键结论。
4. **每轮都给一个“退出/跳过”。** 用户可以点“我还是不懂”，系统降一级提示，而不是卡死。

2.7 为什么用户会信任

信任来源	具体做法
说人话	不用“元认知”“工作记忆”等术语，用“你写到这一步时，前提是什么”
有证据	引用用户原话/原步骤，不凭空判断
会排除	明确说“不是求导，不是计算”，缩小问题
可反驳	诊断卡下方“我不同意”，进入人工/规则复核
不越界	不贴“笨/懒/不认真”的标签，不做医学判断
有下一步	不只说“你弱”，而是“今晚练这 1 道，3 分钟”

3. 科学原理如何变成对话规则

> 这一章不写“脑科学综述”，只写“哪条原理，落到哪句追问”。 每条原理都必须能回答：**如果它成立，系统下一句话应该怎么变？**

3.1 六条原理 → 六条对话规则

原理	一句话解释	对话规则	反例（不要这样做）
先验知识/图式	新知识必须挂在已有结构上，否则记不住	先问“你已经知道什么”，用他的原话做锚点	一上来讲完整解法
预测误差	预期被打破时，注意力和编码最强	先让他预测下一步/结果，再给反例	直接说“正确答案是……”
认知负荷	工作记忆一次只能处理少量新元素	一次只问一个点；长题先分段；把条件外显	一条消息问三个“为什么”
提取练习	从记忆中取出来，比再读一遍更能巩固	让他复述/默写/解释，而不是“看懂了吗”	把答案放在屏幕上让他点头
自我解释	能说出“为什么”才算真的接上	追问“这一步为什么成立”“如果改成……还成立吗”	只问“对不对”
元认知校准	以为自己会，和真的会，常常不一致	每题记信心；诊断后让他评“像不像你”	用“懂了吗”代替校准

3.2 七个能力维度：只做“探针镜头”，不做分数

我们不输出 IQ，也不输出“工作记忆 72 分”。七维只用于**选择下一个问题**：

维度	在单题里的表现	探针示例	不能据此下结论
V 言语理解	读不懂“任意/存在”“恒成立”“至少”	“把‘对任意 x>0 恒成立’改成你自己的话，怎么说？”	不能说他“语文差”
Gf 流体推理	遇到新定义/新结构，抽不出规则	“如果题目把 `ln x` 换成 `e^x`，哪一步会变？”	不能说他“智商低”
Gv 视觉空间	不画图，或画图后看不出关系	“你能把 f(x) 和 f’(x) 的草图分开画吗？”	不能说他“没有空间感”
WM 工作记忆	多条件同时处理时丢条件	“先只做第一步，把 a>0 写在纸上，再往下。”	不能说他“记性差”
Gs 加工速度	会做但慢，考试做不完	“不计时，先做对；慢在哪一步？”	不能说他“笨”
LTS 长时存取	公式/定理想不起来，看到就懂	“遮住公式，写出 `(ln x)'` 和 `(x^a)'`。”	不能说他“基础差”
M 元认知	信心高但做错，或不知道哪个点不会	“这题你觉得自己能对几成？为什么？”	不能说他“态度不好”

使用原则：

1. 七维是**假设生成器**，不是诊断结论；
2. 每个维度至少要有 2 个行为信号，才允许进入候选；
3. 对外只说“这次卡在哪”，不说“你哪项能力低”；
4. 任何维度都不能用于分班、排名、贴标签。

3.3 认知负荷：把“大问题”切成“能回答的小问题”

一道高中导数题，对新手可能是 7–9 个元素同时在线：参数、定义域、导数、驻点、单调性、极值、端点、不等式。工作记忆装不下，就会“看起来会，一做就乱”。

系统的做法：

- **分段**：先只判断“a≤0 时有没有最大值”，不要求解完；
- **外显**：让用户把“a>0”写在纸上/输入框里，减少内存负担；
- **单点**：一轮只处理一个条件，不同时问“定义域和单调性”；
- **降级**：用户说“不知道”，给二选一或具体数值，而不是重复原问题。

3.4 预测误差：让“哦”发生在对话里

直接告诉学生“你漏了 a≤0”，他只会点头。更有效的是让他先预测，再看到预测被打破：

> “你觉得 a=-1 时，f(x) 会不会超过 0？” 学生：“应该不会吧，ln x 增长很慢。” “那你代入 x=10 试试：ln 10 + 10 是多少？” 学生：“12.3……哦，会超过。”

这个“哦”就是预测误差。它比十句“注意分类讨论”更有用。

3.5 提取练习与自我解释：不问“懂了吗”

坏问题	为什么坏	好问题
“懂了吗？”	社交压力下，学生几乎都说懂	“遮住上面，用自己的话复述这一步。”
“哪里不会？”	太抽象，学生真的不知道	“你是卡在‘不知道用哪个公式’，还是‘知道公式但不会代’？”
“为什么？”	对不会的学生，开放式问题像审问	“如果我只改一个条件，你觉得哪一步会变？”
“你再想想。”	没有提供任何支架	“给你一个提示：先看 a=0。现在能判断吗？”

3.6 巩固与迁移：诊断不是终点

一次对话只能解决“看见卡点”，不能保证“长期掌握”。所以系统在诊断卡后接两个动作：

1. **24 小时迁移**：一道同结构、不同情境的题，只测“卡点是否复发”；
2. **7 天变式**：把参数、函数类型或问法换掉，测“是否只会原题”。

这不是“记忆曲线”的装饰，而是把“听懂”与“能做”分开。

3.7 神经可塑性的正确用法

可塑性不是“练得越多越聪明”，而是：

> **有针对性的、有反馈的、有难度的重复，才会改变连接。**

对应产品约束：

- **针对性**：只练被诊断出的那一个点，不整套刷题；
- **有反馈**：每道微练立刻告诉“这一步对不对，为什么”；
- **有难度**：在“跳一跳够得着”的区间，不完全不会，也不一眼就会；
- **有间隔**：24 小时/7 天复测，而不是当晚刷十遍；
- **有情绪安全**：不羞辱、不贴标签；焦虑时先降难度。

3.8 伦理红线

红线	具体做法
不测智商	不输出 IQ、百分位、等级、 “你的工作记忆相当于……”
不下医学诊断	不出现 ADHD、抑郁、学习障碍、感统失调等判断
不排名	不显示“你超过了 83% 的同龄人”
不承诺分数	不说“练完这套提 20 分”
不制造焦虑	不用“你现在已经落后”之类的话术
未成年人保护	默认最小化收集；照片不留存原图；家长视图只看趋势，不看原始作答

3.9 一句话总结

脑科学不是用来讲故事的，是用来决定“下一句话问什么、问多难、什么时候停”的。

4. 诊断引擎：从“答案”到“卡点”

4.1 题目不是一段文本，是一张解法图

系统对每道题生成一张 Solution Graph。节点不是“解题步骤”的流水账，而是“认知动作”：

节点类型	含义	例子
READ	读懂题意、符号、量词	“对任意 x>0 恒成立”
CONCEPT	调用定义/定理/公式	导数与单调性的关系
CONDITION	把文字条件转成式子/图形/分类	a>0 时才有正驻点
STRATEGY	选择整体路径	先分类，再求最值
TRANSFORM	等价变形	ln(1/a)≤1 → a≥1/e
COMPUTE	计算/代数	求导、解不等式
CHECK	校验、边界、反例	检查 a=0、a<0

每个节点带四类元数据：

```
{
  "id": "n4",
  "type": "CONDITION",
  "text": "确认 a>0 时，驻点 x=1/a 才落在定义域 x>0",
  "concept": "parameter_classification",
  "demand": {"V": 3, "Gf": 4, "WM": 3, "Gs": 1, "LTS": 2, "M": 4},
  "prerequisites": ["n2"],
  "common_errors": ["E_NOCASE", "E_DOMAIN"],
  "probe_templates": ["P_CASE_COUNTEREXAMPLE", "P_DOMAIN_CHECK"]
}
```

关键：`demand` 不是“题目难度”，而是“这一步对哪种认知资源的需求”。同一道题，对不同学生，卡在不同节点。



图 4-1 诊断引擎：左为题目解法图，中为学生步骤对齐，右为假设后验与下一探针，底部为卡点卡。

4.2 学生作答不是“对/错”，是一条可对齐的步骤链

系统把用户提交的答案拆成步骤：

```
{
  "attempt_id": "A_1024",
  "steps": [
    {"id": "s1", "text": "f'(x)=1/x-a", "maps_to": "n3", "status": "ok"},
    {"id": "s2", "text": "令 f'(x)=0，得 x=1/a", "maps_to": "n4", "status": "partial",
     "evidence": "未说明 a>0；a=0 时除法无意义"},
    {"id": "s3", "text": "最大值 f(1/a)=ln(1/a)-1", "maps_to": "n5", "status": "ok"},
    {"id": "s4", "text": "a≥1/e", "maps_to": "n6", "status": "ok"}
  ]
}
```

对齐算法三步：

1. **符号归一化**：LaTeX → SymPy 表达式；处理 `ln`、`log`、分数、区间。
2. **步骤匹配**：LLM 把用户步骤映射到图节点；规则校验“节点依赖是否满足”。
3. **第一处偏离**：从入口节点向后走，第一个 `missing/partial/wrong` 且其前置节点都 OK 的位置，就是候选断点。

> 注意：不是找“最后错的那一步”，而是找**最早让整条链断掉的认知动作**。后面可能跟着一串连带错误，那些不是根因。

4.3 卡点分类：L0 – L6

层级	名称	典型信号	第一探针	不要误判为
L0	读题/语义	把“任意”读成“存在”；不知道符号含义	“把这句话改成自己的话”	粗心
L1	概念/定理	不知道 `f'(x)=0` 是必要不充分；公式记错	“遮住公式，写出……并解释适用条件”	基础差
L2	条件转译	不会把文字条件变式子；漏参数分类；漏定义域	“当 a=0 时，这条式子还成立吗？”	不会做
L3	策略	知道所有知识点，但不知道先做什么	“如果只允许你走一步，你走哪步？为什么？”	没思路
L4	执行	策略对，但算错/丢步骤/符号错	“这一步再算一遍，只检查符号”	粗心
L5	校验/元认知	不检查边界；信心高但错；不知道哪里不会	“你觉得自己这题能对几成？为什么？”	态度问题
L6	资源/状态	时间不够、焦虑、注意力	“先不计时，做第一步”	学习能力差

判定原则：

- 至少 2 个独立信号，才允许定层；
- 能落到更具体节点，就不只给 L 层；
- L6 不进入教学内容，只做降难度/休息建议，必要时转介；
- 任何层级都不使用“笨、懒、不认真”等标签。

4.4 假设空间：一次只比较 2 – 3 个可能

系统不追求“全自动诊断”，而是维护一个小假设集：

```
{
  "session_id": "S_9001",
  "candidates": [
    {"hypothesis": "H_L2_CASE", "label": "漏参数分类", "prior": 0.42, "posterior": 0.42},
    {"hypothesis": "H_L1_DERIV", "label": "导数与极值概念不清", "prior": 0.25, "posterior": 0.25},
    {"hypothesis": "H_L4_CALC", "label": "解不等式计算错误", "prior": 0.18, "posterior": 0.18}
  ]
}
```

先验来自：

- 题目节点的 `common_errors`；
- 用户历史（如果存在）；
- 用户自述“卡在哪”；
- 答案/步骤的初步对齐结果。

4.5 探针选择：问“最有区分度”的那个

每个探针都预先标注：它能把哪些假设区分开。

探针类型	例子	主要区分
P_COUNTEREXAMPLE	“a=-1 时，f(x) 会不会超过 0？”	L2 分类 vs L1 概念
P_DOMAIN_CHECK	“x=1/a 在 a<0 时是什么符号？”	L2 条件转译 vs L4 计算
P_EXPLAIN_STEP	“为什么 f’(x)=0 能说明是最大值？”	L1 概念 vs L3 策略
P_NEXT_STEP	“只允许走一步，你走哪步？”	L3 策略 vs L4 执行
P_CONFIDENCE	“这题你能对几成？为什么？”	L5 元认知
P_RETRIEVAL	“遮住公式，写出 (ln x)’”	L1/LTS 提取失败

选择规则：

1. 只在 top 2–3 假设之间选；
2. 优先选**预期信息增益最大**的探针；
3. 如果两个假设用同一个探针无法区分，换一个；
4. 一轮只问一个；
5. 连续两次“不知道”，降级为二选一/具体数值/半成品填空。

4.6 置信度更新与停止条件

用简单的对数几率更新，不做“黑箱打分”：

```
logit(H_i | e) = logit(H_i) + log LR(e | H_i)
```

每个探针给出一个离散结果（能/不能/不确定/拒绝），对应预设的似然比。示例：

探针结果	对 H_L2（分类）	对 H_L1（概念）	对 H_L4（计算）
能正确判断 a=0 不成立，但没主动分类	×3.0	×0.8	×1.0
认为 a<0 时仍有最大值	×0.4	×2.5	×0.6
计算 ln(1/a)≤1 出错	×0.8	×0.7	×4.0

停止条件（满足任一）：

- 最高后验 ≥0.80，且与第二名差距 ≥0.25；
- 已问 6 轮；
- 用户连续两次说“不知道”，且降级后仍无法判断；
- 用户主动要求“直接告诉我”。

置信度展示：

后验	文案	行为
≥0.80	“我比较确定”	给卡点卡 + 微干预
0.55–0.79	“我倾向于”	给卡点卡 + 一个验证问题
<0.55	“目前还不能确定”	给 2–3 个可能，让用户选/继续问

4.7 诊断卡：不是“报告”，是“下一动作”

诊断卡固定六块：

1. **卡点**：L 层 + 具体节点（一句话）；
2. **证据**：引用用户的原话/步骤；
3. **排除**：明确说“不是什么”；
4. **置信度**：高/中/低 + 数字；
5. **下一练**：3 分钟能完成的一个动作；
6. **验证**：24 小时/7 天的一道新题。

反例（不要这样写）：

> “你的函数与导数基础薄弱，建议加强练习。”

这是标签，不是诊断。它没有证据、没有排除、没有下一步。

4.8 失败与降级

失败	表现	降级
OCR 错	公式识别成别的	让用户确认/手改关键符号；保留原图不展示
解法图缺失	新题不在 100 道种子库	用 LLM 生成草图 + SymPy 校验；标记“探索模式”，置信度上限 0.65
多卡点	同时缺概念和计算	只选“最早断点”，其余记入“后续观察”，不一次讲完
用户拒绝回答	只想要答案	解释规则；给“看答案”入口，但退出学习模式，不记录诊断
用户情绪	焦虑/自我否定	停止追问，先共情；建议休息/找老师，不做心理判断
答案泄漏	模型想给完整解	出口过滤器拦截，重写为提示；记录 `answer_leak` 事件

4.9 一个最小可用的解法图示例

以 `f(x)=ln x - a x` 为例：

```
n1 READ      读懂“对任意 x>0，f(x)≤0 恒成立”
n2 CONCEPT   知道要研究 f 的最大值
n3 COMPUTE   求导 f'(x)=1/x-a
n4 CONDITION 分 a≤0 / a>0；确认 a>0 时驻点 x=1/a 在定义域
```

```
n5 CONCEPT   a>0 时 f''<0，x=1/a 是最大值
n6 COMPUTE   解 ln(1/a)-1≤0
n7 CHECK     验证 a=1/e 时等号成立；a<1/e 时反例
```

依赖：`n1→n2→n3→n4→n5→n6→n7`。

学生答案对齐后，`n4` 是 `partial`，`n1–n3` 都 OK，`n5–n7` 结果对但建立在未验证前提上。因此根因是 `n4`，不是 `n6`。

这就是“错哪里了”与“卡在哪”的区别：错在结果，卡在前提。

5. Prompt 与 Agent：让“追问”可测试

5.1 四个运行时角色

角色	职责	输入	输出	是否用 LLM
Parser	OCR 纠错、LaTeX 归一化、题目/答案分离	图片/文本	结构化题目、用户步骤	是（多模态）
Diagnostician	生成/读取解法图、对齐步骤、维护假设后验	题目图 + 用户步骤	candidates[]、first_divergence	是 + 规则
Socratic Coach	选探针、生成一次一问的追问、降级提示	假设集 + 探针库	下一句话 + JSON	是
Verifier	SymPy 校验、答案泄漏检测、前后一致性	候选回答/诊断	pass/fail + 修正建议	规则 + 小模型

内容侧还有一个离线角色 Curator：把教材、课标、错因库、提示库整理成可检索的 chunk；它不在用户对话链路里。

5.2 状态机：不是“多 Agent 聊天”

```
INGEST
  → PARSE
  → GRAPH
  → ALIGN
  → HYPOTHESIZE
  → PROBE ──┐
  ↑         │ (最多 6 轮)
  └─────────┘
  → CONFIRM
  → INTERVENE
  → VERIFY
  → CLOSE
```

规则：

- 状态转移由代码控制，LLM 只能在 PROBE 内选择“问什么/怎么问”；
- 每次 handoff 写 from/to/reason/state_version；
- PROBE 每轮必须更新 candidates[].posterior；
- 达到停止条件才能进 CONFIRM；
- CONFIRM 必须让用户评分“像不像你”，低分回到 HYPOTHESIZE；
- 任何状态失败，降级到“模板卡 + 人工复核”，不静默重试。

5.3 系统提示词（核心版）

```
你是 SynapseMind 的单题诊断教练。你的任务不是讲完这道题，而是在最多 6 轮内，
找到学生“最早断掉的那个认知动作”，并让他自己确认。

【硬约束】
1. 一次只问一个问题。不要在一句话里问两个“为什么”。
2. 前 3 轮不出现最终答案、完整解法、关键结论。
3. 先引用学生的原话/原步骤，再提问；不要凭空判断。
4. 学生说“不知道”时，降一级：二选一 → 具体数值 → 半成品填空。
5. 不贴标签：不说“你粗心/笨/懒/基础差/不认真”。
6. 不做医学/智商判断；不承诺分数。
7. 每轮输出必须是 JSON，字段见 schema；不要输出 JSON 以外的字。
8. 如果你不确定，action=ask，并在 evidence 里写“待验证”，不要硬下结论。

【当前状态】
problem_graph: {{graph}}
attempt_steps: {{steps}}
candidates: {{candidates}}
turn: {{turn}}
max_turns: 6

【本轮任务】
选择最能区分 top-2 假设的一个探针；生成一句学生能回答的问题。
如果最高后验 ≥0.80 且领先 ≥0.25，action=diagnose。
如果已到第 6 轮，action=diagnose 或 stop。
```

5.4 输出 Schema

```
{
  "type": "object",
  "required": ["turn", "action", "question", "target_hypothesis", "confidence"],
  "properties": {
    "turn": {"type": "integer", "minimum": 1, "maximum": 6},
    "action": {"enum": ["ask", "hint", "confirm", "diagnose", "intervene", "stop"]},
    "question": {"type": "string", "maxLength": 120},
    "options": {"type": "array", "items": {"type": "string"}, "maxItems": 4},
    "target_hypothesis": {"type": "string"},
    "probe_id": {"type": "string"},
    "hint_level": {"type": "integer", "minimum": 0, "maximum": 3},
    "evidence": {"type": "array", "items": {"type": "string"}, "maxItems": 3},
    "confidence": {"type": "number", "minimum": 0, "maximum": 1},
    "should_end": {"type": "boolean"}
  },
  "additionalProperties": false
}
```

为什么用 schema：

- 追问可以被单测：给定假设集，断言输出必须包含某个 `probe_id`；
- 前端可以稳定渲染选项；
- 答案泄漏可以在 schema 层拦截（没有 `solution` 字段）；
- 模型升级时，回归测试有契约。

5.5 好问题 / 坏问题

场景	坏问题	好问题	为什么

场景	坏问题	好问题	为什么
漏参数分类	“你考虑 a<0 了吗？”	“a=-1 时，f’(x)=1/x+1 有零点吗？”	让事实自己说话
概念不清	“极值的必要条件是什么？”	“f’(x)=0 能保证是最大值吗？举个反例。”	用反例检验
不会起步	“你觉得该怎么做？”	“只允许走一步：求导、画图、还是分类？选一个。”	降低选择负担
计算错	“再算一遍。”	“只检查最后一步的符号，`ln(1/a)≤1` 推出什么？”	定位到具体动作
元认知	“你有把握吗？”	“这题你能对几成？为什么？”	校准而非情绪

5.6 答案泄漏的四道闸

闸	位置	做法
1. 检索	RAG	`chunk_type=solution` 在学习模式硬过滤：只召回 `hint/concept`
2. Prompt	生成	schema 没有 `solution` 字段；系统提示禁止最终答案
3. 出口	输出	5-gram 与标准答案重合 >0.6 拦截；SymPy 等价检测
4. 回归	评测	100 条红队 prompt，答案泄漏率必须为 0

注意：只靠“提示词说不要给答案”一定会破。答案必须从数据、结构、出口三层一起锁。

5.7 MCP：把能力暴露给 IDE，而不是再写一套 API

MCP Server 暴露 6 个工具：

工具	作用	权限
`get_problem_graph`	按题目 ID 取解法图	读
`align_attempt`	对齐学生步骤	读
`search_concept`	检索概念卡/提示	读
`verify_math`	SymPy 校验表达式/区间	读
`next_probe`	取候选探针	读
`record_evidence`	写诊断证据	写（双人复核）

用途：

- 线上 Agent 调工具；
- 教研在 Cursor/Claude Code 里调同一套工具，批量校对解法图；
- 不需要维护第二套内部 API。

5.8 Agent Skills：把“怎么追问”变成可发布资产

目录：

```
skills/
  diagnostic-review/
    SKILL.md
    skill.yaml
    scripts/check_evidence.py
    references/probe_patterns.md
    evals/golden.jsonl
  hint-policy/
    SKILL.md
    skill.yaml
    scripts/check_answer_leak.py
```

`diagnostic-review` 做两件事：

1. 检查诊断卡是否有证据、排除项、置信度；
2. 检查是否把“假设”写成了“结论”。

`hint-policy` 做两件事：

1. 检查提示层级是否逐级上升；
2. 检查是否在 3 轮内泄漏答案。

这两个 Skill 可以在 CI 里跑，也可以被 IDE 调用。

5.9 模型路由与成本

任务	模型档位	占比	原因
OCR 纠错/格式归一	小模型/专用 OCR	30%	便宜、可批处理
步骤对齐/分类	7B–14B	40%	结构化、低创造性
追问生成	32B	25%	需要语言自然度
难题诊断/红队	旗舰	5%	少数难例

单次诊断 4–6 轮，目标成本 ≤¥0.8；超出预算时，降级为“规则探针 + 模板话术”，不牺牲答案安全。

5.10 失败时的兜底

- LLM 超时：返回上一轮缓存 + “我先问一个更简单的问题”；
- JSON 解析失败：重试 1 次，仍失败用模板；
- 置信度 <0.55：不硬判，给 2–3 个可能；
- 用户情绪激烈：停止追问，输出“我们先停一下”，转人工/教师；
- 题目超纲：明确说“这道题当前版本只能做步骤级提示”，不假装能诊断。

6. RAG 与内容：让追问有据可查

6.1 知识库不是“教材切片”，是五类卡片

类型	内容	典型字段	用途
`concept`	概念卡：定义、适用条件、反例	`concept_id`, `grade`, `prereq`	解释“为什么”
`graph`	题目解法图	`problem_id`, `nodes`, `edges`	对齐与定位
`error`	错因卡：L0 - L6 + 典型信号	`error_id`, `layer`, `signals`	生成假设
`hint`	提示卡：0 - 3 级，逐级上升	`hint_id`, `level`, `target_node`	降级支架
`transfer`	迁移题：同结构/变式	`item_id`, `root_node`, `answer_hash`	24h/7d 验证

关键设计：chunk 不是固定 500 字，而是**一张卡 = 一个语义单元**。这样检索结果可以直接作为“提示”，而不是一段需要再总结的教材。

6.2 文档 ETL

```
PDF/图片/网页
  → Tika/PaddleOCR
  → 结构识别（标题/公式/表格）
  → LaTeX 归一化
  → 人工校对（关键题）
  → 卡片化（concept/graph/error/hint）
  → bge-m3 向量化
  → PostgreSQL + pgvector
```

数学公式处理：

- `ln x`、`\ln x`、`log_e x` 统一为 `ln(x)`；
- 区间 `[1/e,+∞)` 统一为 `Interval(1/e, oo)`；
- 用 SymPy 做等价校验，避免“看起来一样、其实不同”。

6.3 切分策略

内容	切分	原因
概念卡	按“定义/条件/反例/常见误用”四段	追问时只取需要的一段
解法图	按节点，不切边	保留依赖关系
错因卡	一卡一因	假设空间要可枚举
提示卡	按 level 切	0 - 3 级逐级调用

内容	切分	原因
教材	按小节 + 公式块	保留上下文

每块 200–480 token，重叠 60–80 token；解法图节点不切。

6.4 检索：混合召回 + 重排

一次追问的 query 不是“题目原文”，而是：

```
题目：ln x - a x ≤ 0 恒成立
学生步骤：令 f'(x)=0 得 x=1/a
候选假设：漏参数分类 / 概念不清
目标：找“驻点存在性”的概念卡 + 反例提示
```

检索流程：

1. **查询增强**：把题目、学生步骤、候选假设拼成 query；必要时 HyDE 生成“假设性解释”，但只在诊断阶段用，不直接展示。
2. **混合召回**：BM25 top 40 + dense top 40。
3. **RRF 融合**：`score = Σ 1/(60+rank_i)`。
4. **重排**：bge-reranker top 50 → 8。
5. **硬过滤**：`acl_level`、`chunk_type != solution`、年级/学科匹配。
6. **阈值**：低于 0.35 不强行引用，改为“通用提示”。

6.5 pgvector 配置

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE kb_chunk (
  id            BIGSERIAL PRIMARY KEY,
  chunk_type    TEXT NOT NULL,          -- concept/error/hint/graph
  title         TEXT NOT NULL,
  content       TEXT NOT NULL,
  metadata      JSONB NOT NULL,
  embedding     vector(1024) NOT NULL,
  acl_level     SMALLINT NOT NULL DEFAULT 1,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_kb_chunk_hnsw
  ON kb_chunk USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 200);

CREATE INDEX idx_kb_chunk_type ON kb_chunk (chunk_type);
```

查询时设置 `SET LOCAL hnsw.ef_search = 80;`，在召回率与延迟之间取平衡。

6.6 内容生产：100 题，30 题深标

MVP 不追求“全题库”，追求“可诊断的密度”。

层级	数量	标注内容	用途
种子题	100	题目、答案、解法图骨架、知识点	覆盖高频题型
深标题	30	每个节点的 `demand`、`common_errors`、`probe_templates`	支撑高质量诊断
金标准对话	100 条	学生步骤 + 教师卡点 + 好/坏追问	评测
迁移题	60	每个深标题 2 道同构/变式	24h/7d 验证

标注流程：

1. 一位教研写解法图；
2. 第二位教研独立标卡点与探针；
3. 不一致的题进入“争议集”，第三人裁决；
4. 只有双人一致的题，才允许进入“高置信”诊断。

质量门槛：30 道深标题的节点一致率 ≥0.80，探针一致率 ≥0.75；达不到就缩范围，不硬上。

6.7 提示卡：0 - 3 级，不许跳级

级别	形式	例子
0	元认知提示	“你先说说，你写到这一步时在算什么？”
1	方向提示	“先别求最大值，先看 a≤0 时有没有最大值。”
2	具体提示	“a=-1 时，f’(x)=1/x+1，它有零点吗？”
3	半成品	“f’(x)=1/x-a。当 a>0 时，令 f’(x)=0，得 x=____。”

规则：用户说“不知道”一次，降一级；连续两次，给半成品；不允许从 0 直接跳到 3。

6.8 引用与可追溯

每条提示/解释都带：

- `source_id`：概念卡/教材页码；
- `source_version`：内容版本；
- `why`：为什么召回它（对应哪个假设/节点）。

诊断卡里的“为什么不是计算错误”，必须能链到一条 `error` 卡和一条用户证据。没有来源的“我觉得”，不允许进入诊断。

6.9 内容侧的 SEO/GEO

这不是主产品，但能自然体现 skill.txt 第 7 项：

- 公开页 `/q/{problem_id}/breakpoints`：这道题的 3 个常见卡点（不展示完整答案）；
- 概念页 `/concept/{slug}`：定义、适用条件、反例、1 道自测；
- 结构化数据：`LearningResource` + `FAQPage`；
- `llms.txt` + sitemap 分片；
- 监测 AI 搜索引用：答案前置、事实密度、来源日期。

公开页的作用不是“引流”，而是让教研内容被检索、被引用、被纠错，反哺诊断库。

7. 技术架构：为“一次对话”设计，不为“大平台”设计

7.1 四个部署单元，七个服务边界

MVP 只跑 4 个进程，避免 3 个人被 Nacos/K8s 拖死；但边界按 7 个服务设计，后续可拆。

部署单元	包含的服务边界	职责
gateway	API Gateway	鉴权、限流、SSE 转发、灰度
core	session、problem、diagnosis、evidence	会话、题目图、对齐、假设、诊断卡
ai	ai-orchestrator、parser、coach、verifier	OCR、LLM 调用、追问生成、校验
content	knowledge、content-admin	知识库、提示卡、迁移题、标注后台

目标 7 服务：

```
gateway
session-service
problem-service
diagnosis-service
knowledge-service
evidence-service
ai-orchestrator
```

拆分触发（写进 ADR，不拍脑袋）：

- 日均诊断 > 20 万次；
- AI 调用与业务事务的资源画像差 > 3 倍；
- 周发布 > 2 次且互相阻塞；
- 故障域需要隔离（AI 挂了，诊断卡仍可出）。

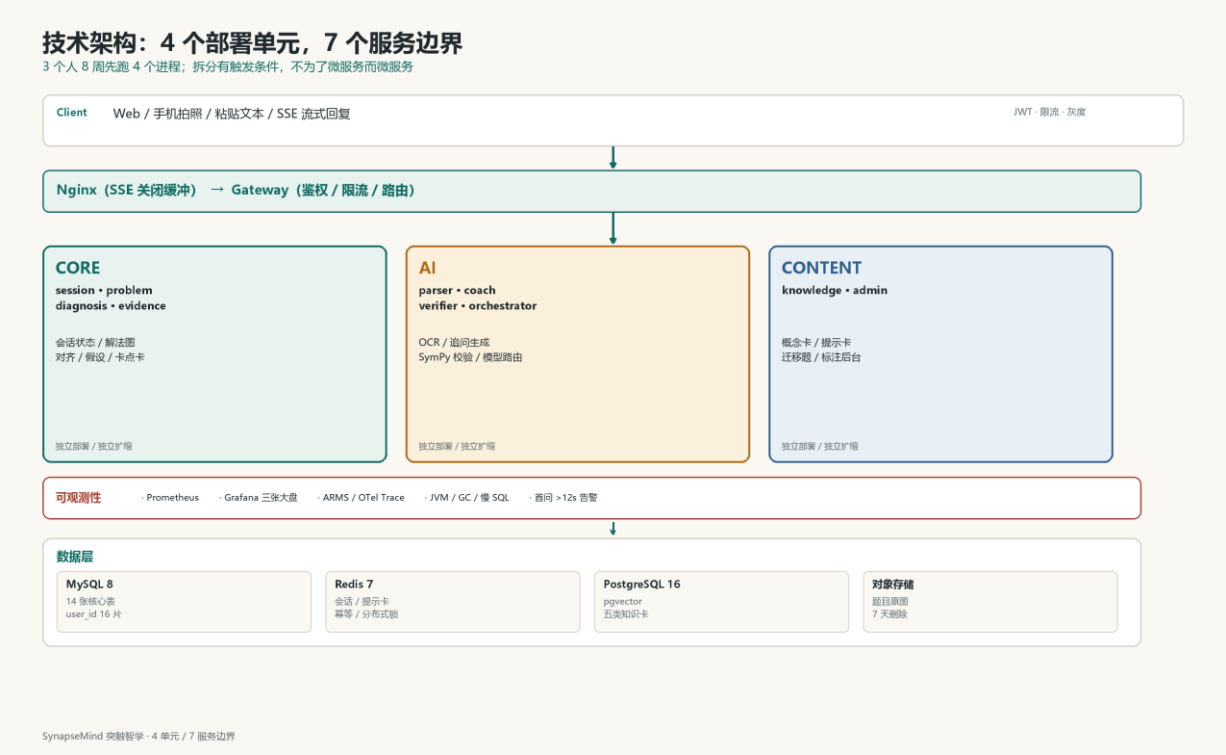


图 7-1 4 个部署单元、7 个服务边界；MySQL/Redis/pgvector 数据层；可观测性横贯全链路。

7.2 核心表

#	表	作用	关键字段	索引
1	t_user	用户	id, grade, status	PK
2	t_problem	题目	id, subject, stem, answer_hash, graph_id	(subject, graph_id)
3	t_solution_node	解法图节点	id, problem_id, type, demand_json, prereq	(problem_id, type)
4	t_attempt	一次作答	id, user_id, problem_id, source, created_at	(user_id, created_at)
5	t_attempt_step	作答步骤	id, attempt_id, seq, latex, maps_to, status	(attempt_id, seq)
6	t_dialogue_session	诊断会话	id, attempt_id, state, turn_count, mode	(attempt_id)
7	t_dialogue_turn	每轮对话	id, session_id, role, content, probe_id, confidence	(session_id, seq)
8	t_hypothesis	假设后验	id, session_id, code, prior, posterior	(session_id, posterior)
9	t_evidence	证据	id, session_id, type, quote, node_id	(session_id)
10	t_intervention	微干预	id, session_id, concept_id, task_id, status	(session_id)
11	t_transfer_check	迁移验证	id, user_id, root_node, item_id, result	(user_id, root_node)

#	表	作用	关键字段	索引
12	t_concept_card	概念卡	id, slug, content, version	(slug)
13	t_hint	提示卡	id, node_id, level, content	(node_id, level)
14	t_llm_call	LLM 调用审计	id, session_id, model, tokens, cost, latency	(session_id, created_at)

分片：t_attempt、t_attempt_step、t_dialogue_turn、t_evidence 按 user_id 16 片；MVP 单库 + 月分区，ShardingSphere 配置就绪但不路由。

索引原则：

- 所有“按会话取”的查询走 (session_id, seq)；
- 所有“按用户取最近”的查询走 (user_id, created_at desc)；
- t_llm_call 按月分区，保留 90 天，聚合后归档。

7.3 一次诊断的调用链

```
Client
  → Nginx (SSE, no buffering)
  → Gateway (JWT, rate limit)
  → Session (create session)
  → Parser (OCR/LaTeX)
  → Problem (load graph)
  → Diagnosis (align steps)
  → AI Orchestrator (hypotheses + next probe)
  → Knowledge (RAG: concept/hint)
  → Coach (generate question)
  → Verifier (answer leak / SymPy)
  → SSE stream back
```

耗时目标：

阶段	P95
图片 OCR	<3s
解法图加载	<100ms（Caffeine）
步骤对齐	<2s
下一问生成	<3s
首条追问（端到端）	<8s

7.4 缓存与并发

缓存	内容	TTL	说明

缓存	内容	TTL	说明
Caffeine	解法图、概念卡	30min	本地热数据，抗 Redis 故障
Redis	会话状态、最近 6 轮	2h	SSE 断线可恢复
Redis	提示卡	6h	按 `node_id + level`
Redis	幂等键	10min	防止重复提交
布隆过滤器	不存在的 `problem_id`	—	防穿透

并发控制：

- 同一 `session_id` 的追问串行（Redisson 锁，TTL 30s）；
- 同一用户同时最多 2 个会话；
- LLM 调用用信号量隔离，避免慢请求拖垮整个 ai 单元；
- SSE 断线重连从 `last_event_id` 续传。

7.5 可观测性

每次对话是一个 trace：

```
trace_id = session_id
span: gateway → session → parser → diagnosis → llm → rag → verifier
```

核心指标：

指标	目标	告警
首条追问 P95	<8s	>12s 5min
诊断完成率	≥70%	<55% 1h
平均轮数	4–6	>7
答案泄漏	0	任何一次
LLM 错误率	<1%	>3% 5min
单次成本	≤¥0.8	>¥1.2
用户“说中了”	≥70%	<60% 1d

Grafana 三张大盘：

1. **业务**：进入 → 交题 → 首问 → 诊断卡 → 微干预 → 24h 验证；
2. **AI**：模型延迟、token、成本、JSON 失败率、RAG 命中率、泄漏拦截；
3. **系统**：JVM、GC、连接池、Redis 命中率、MySQL 慢 SQL。

7.6 部署

```
docker-compose:
  nginx
  gateway
  core
  ai
  content
  mysql
  redis
  postgres(pgvector)
  prometheus
  grafana
```

- **Dockerfile**：多阶段构建，运行镜像 <250MB；
- **Nginx**：`proxy_buffering off`、`X-Accel-Buffering: no`、SSE 超时 300s；
- **Serverless**：OCR 批处理、PDF 导出、离线评测跑在 FC，避免占常驻内存；
- **K8s**：Phase 2，按 7 服务拆，HPA 按 CPU + 队列长度。

7.7 安全与合规

风险	措施
学生照片含隐私	原图加密存储 7 天；只留 OCR 文本；默认不展示原图
未成年人	监护人同意；最小化收集；不做画像广告
Prompt 注入	题目/答案中的指令隔离；系统提示优先级；输出 schema
答案泄漏	四道闸；红队 100 条；线上采样
模型幻觉	SymPy 校验；引用必须带 `source_id`；置信度低不硬判
数据删除	用户可一键删除会话与证据；30 天内彻底清理

7.8 为什么不是“一上来微服务”

3 个人、8 周，如果先搭 Nacos + Gateway + 7 个服务 + K8s，真正做诊断的时间不到 3 周。正确顺序：

1. 先把 30 道深标题的解法图和探针跑通；
2. 用一个 core 进程 + 一个 ai 进程上线；
3. 当 AI 调用量/故障域/发布节奏真的需要时，再按 7 服务拆。

架构判断力不是“会用多少中间件”，而是知道什么时候不用。

8. skill.txt 逐条映射：每个能力落在哪个文件、哪次演示

> 原则：**不堆名词。** 每一项都要回答：在本项目里，哪个模块用它？演示时看什么？怎么证明不是 PPT 工程？

8.1 开发技术 1-7

#	能力	本项目落点	可演示证据	验收指标
1	AI 辅助编程；Java/Python/Go；Hutool、Lombok；代码规范	Java 21 主栈（gateway/core/content）；Python 做 OCR 后处理、离线评测、标注 ETL；Go 写 SSE 事件采集 sidecar（可选）；Hutool 做 LaTeX 清洗、JSON 树、周区间；Lombok 全量；Spotless + Checkstyle + ArchUnit	`collector/`、`eval/`、`etl/` 目录；CI 报告；AI 生成单测的 commit；Code Review 记录	单测覆盖率 ≥60%；Checkstyle 0 error；核心规则手写率 100%
2	微服务、Spring Cloud、MyBatis、单体拆微服务治理	Gateway + Nacos + OpenFeign + Sentinel；4 单元 → 7 服务边界；MyBatis-Plus 分页/逻辑删除；ArchUnit 禁止跨层依赖；拆分 ADR	`docker-compose.yml`；Feign 契约；Nacos 快照；ADR-001；灰度发布记录	4 单元一键起；服务边界依赖 0 违规；拆分触发条件有数据
3	MySQL 设计、SQL 调优、分库分表、索引优化、Druid	14 张核心表；`user_id` 16 片；联合索引/覆盖索引；3 条关键 SQL 优化；Druid StatFilter 慢 SQL 日报；月分区	`schema.sql`；`sharding-config.yaml`；EXPLAIN 前后对比；慢 SQL Top20	诊断会话查询 P95 <50ms；慢 SQL <10 条/天
4	Redis、Caffeine 多级缓存、分布式锁、雪崩/穿透/击穿、哨兵	Caffeine 解法图；Redis 会话/提示卡/幂等；Redisson 会话锁；布隆过滤器；逻辑过期 + 随机 TTL；Redis Sentinel 哨兵集群	Grafana 命中率；Redisson 配置；故障演练：Redis 挂 30s 仍可出诊断卡	缓存命中率 ≥85%；故障演练通过
5	Docker、Nginx、Dockerfile、Serverless	多阶段镜像；Compose 一键起；Nginx SSE 关闭缓冲 + 限流；OCR/PDF 导出跑 FC	`Dockerfile`；`nginx.conf`；FC 日志；镜像大小对比	镜像 <250MB；SSE 不中断；冷启动 <3s
6	Prometheus + Grafana + ARMS、全链路追踪、JVM 诊断大盘	每次对话一个 trace；LLM span；三张大盘；5 条 SLO；ARMS Agent；一次 OOM/GC 复盘	大盘 JSON；告警规则；Trace 截图（含 LLM span）；OOM 复盘	首问 P95 <8s；告警 5min 内触发

#	能力	本项目落点	可演示证据	验收指标
7	SEO 与 GEO、结构化内容、TDK、收录与 AI 推荐排名	公开“题目卡点库”：`/q/{id}/breakpoints`、`/concept/{slug}`；SSR；TDK 模板；JSON-LD `LearningResource`/`FAQPage`；`llms.txt`；sitemap 分片；收录监测；AI 引用监测	页面 URL；Rich Results 校验；Search Console；llms.txt；AI 搜索引用截图	100 页被收录；20 个概念页被 AI 引用/提及

8.2 AI 应用开发 1-4

#	能力	本项目落点	可演示证据	验收指标
8	Spring AI、LangChain4j；RAG、工具调用、MCP、多 Agent	Spring AI 编排；LangChain4j 结构化输出；Parser/Diagnostician/Coach/Verifier 四角色状态机；RAG 五类卡片；6 个 MCP 工具	一次完整诊断 trace；MCP Inspector 录屏；Agent 状态转移图；`handoff` 日志	追问 JSON 合法率 ≥99%；状态机回放一致
9	Prompt 工程；PGvector；ETL；检索；查询增强	系统提示 + JSON Schema；pgvector HNSW；文档 ETL（Tika/PaddleOCR）；BM25 + dense + RRF + rerank；结构感知切分；查询改写；引用校验	`kb_chunk` 表；加/不加 rerank 对比；引用日志；prompt 版本记录；100 条 golden	recall@8 ≥0.85；引用可追溯 100%
10	Cursor / Claude Code / GitHub Copilot 全栈开发；MCP Server / Agent Skills	自建 MCP Server：`diagnostic-review`、`hint-policy` 两个 Skill；CI 回归；canary 发布	Cursor/Claude Code 里经 MCP 取解法图、校对一个节点；`skills/` 目录；canary 记录	Skill 在 CI 跑通；MCP 工具 6/6 可用
11	Vibe Coding、SDD、Harness、Spec-Kit/OpenSpec	spec 先行：`specs/dialogue-state.md`、`specs/diagnosis.schema.json` 为唯一真源；OpenSpec 变更单；golden 回归；回滚演练	spec 目录；变更单；回归报告；回滚记录	需求变更 100% 有 spec；回滚 <10min

8.3 覆盖度自检

能力域	覆盖	说明
Java/Python/Go	☑	Java 主栈；Python ETL/评测；Go 采集 sidecar
Hutool/Lombok	☑	LaTeX 清洗、JSON 树、周区间；Lombok 全量
Spring Cloud	☑	Gateway/Nacos/Feign/Sentinel；4→7 边界
MySQL/分表/Druid	☑	14 表、16 片、3 条 SQL、慢 SQL
Redis/Caffeine/锁/三防	☑	多级缓存、会话锁、布隆、逻辑过期
Docker/Nginx/Serverless	☑	多阶段、SSE、FC
监控/追踪/JVM	☑	三张大盘、5 条 SLO、一次复盘
SEO/GEO	☑	100 页卡点库、JSON-LD、llms.txt

能力域	覆盖	说明
Spring AI/LangChain4j/RAG/MCP/多 Agent	☑	四角色状态机 + 五类卡片 + 6 工具
Prompt/PGvector/ETL	☑	Schema、HNSW、切分、改写、引用
Cursor/Claude Code/MCP/Skills	☑	自建 Server + 两个 Skill
SDD/Harness/Spec-Kit	☑	spec 先行、golden、回滚

诚实标注：

- Go sidecar、K8s、16 片真实分库、ARMS 全量接入属于 **Phase 2**；MVP 有配置/脚本/演练，但不假装已跑过大规模流量。
- SEO/GEO 在 MVP 只做 100 页公开卡点库，不追求全站内容矩阵。
- MCP 的写工具（`record_evidence`）需要双人复核，MVP 只对教研开放。

8.4 面试时怎么讲

> “这个项目不是‘我用了 Spring Cloud 和 RAG’，而是： 为了让学生一次只回答一个问题，我把追问做成状态机，LLM 只能在 `PROBE` 状态里选探针； 为了防止 AI 变成搜题，我把答案锁在检索、schema、出口、回归四道闸里； 为了让‘卡点’可追溯，每张诊断卡都带证据、排除项和置信度； 为了 3 个人 8 周能做出来，我只跑 4 个部署单元，但边界按 7 个服务设计。 这些取舍，比堆中间件更能说明我会做工程。”

8.5 90 秒陈述

> “一个学生发来一道题和自己的答案。传统产品给答案，AI 家教讲整题，错题本只记‘错了’。 我们做的是：先拆解法图，再把他的步骤贴上去，找第一处偏离；然后用 2–6 个具体问题，让他自己发现‘我卡在参数分类，不是不会求导’；最后给一张卡点卡、3 分钟微干预、一道迁移题。 技术上，规则定骨架，LLM 只负责追问和措辞；RAG 只召回提示卡，不召回答案；多 Agent 是状态机，不是聊天室。 MVP 只做高中数学函数与导数、100 道题、30 道深标；3 个人 8 周；先证明‘诊断与双教师一致率 ≥75%’，再谈规模。”

8.6 简历写法（标明设计/目标）

- **S**：学生遇到错题，只得到“对/错”或完整解答，不知道自己具体卡在哪个认知动作，导致重复无效刷题。
- **T**：设计一个单题诊断引导系统，输入题目 + 学生答案，输出可解释的卡点与下一步训练。
- **A**：定义 L0–L6 卡点分类与解法图；设计“规则/贝叶斯 + LLM 追问”的状态机；实现 RAG 五类卡片与答案四道闸；用 4 单元/7 服务边界控制 3 人 8 周范围；建立 100 条 golden 对话评测。

- **R（目标）**：诊断与双教师一致率 ≥75%；用户“说中了” ≥70%；平均 4–6 轮；答案泄漏 0；单次成本 ≤¥0.8。

技术关键词（按 JD 取舍）：Java 21、Spring Boot 3、Spring Cloud、Nacos、Gateway、Sentinel、OpenFeign、MyBatis-Plus、MySQL、Redis、Caffeine、Redisson、PostgreSQL/pgvector、Spring AI、LangChain4j、RAG、MCP、多 Agent、Docker、Nginx、Prometheus、Grafana、Next.js、SEO/GEO。

9. 可行性与 MVP：3 个人 8 周，先证明“说得准”

9.1 MVP 范围

项	决策	为什么
学科	高中数学“函数与导数”	解法图可控；SymPy 可验证；高频
题型	参数范围、单调性、极值、零点个数、恒成立	五种题型覆盖大多数导数题
题量	100 道种子题，30 道深标	深标比数量重要
输入	文本 + 拍照（OCR）	学生真实入口
输出	卡点卡 + 3 分钟微干预 + 1 道迁移题	一次只解决一个点
用户	高一至高三	不做小学/大学
团队	3 人：1 全栈/后端、1 AI、1 教研/产品	内容与工程并行
周期	8 周	比“大平台”少 2 周，砍掉非核心

9.2 八周计划

周	目标	交付	退出条件
W1	定契约	解法图 schema、卡点 L0–L6、诊断卡 JSON、10 道样例题	两位教研对 10 道题的节点一致率 ≥0.8
W2	走通单题	文本输入 → 解法图 → 步骤对齐 → 卡点卡（模板）	3 道题端到端可演示
W3	追问	探针库 v0、状态机、一次一问、降级	5 轮内能区分 2 个假设
W4	多模态	拍照 OCR、LaTeX 归一化、人工纠错入口	10 张手写题识别可用率 ≥80%
W5	RAG	概念卡/提示卡入库、pgvector、混合检索	recall@8 ≥0.80
W6	微干预	3 分钟讲解模板、同构题、迁移题	用户能独立完成同构题 ≥60%
W7	评测	100 条 golden、双教师标注、红队 50 条	一致率 ≥70%，泄漏 0
W8	封测	40–60 人 beta、埋点、复盘	完成率 ≥70%，说中了 ≥65%

砍掉什么：

- 不做完整课程、不做题库、不做社交、不做家长端复杂报表；
- 不做 7 服务物理拆分、不做 K8s、不做全量 SEO；
- 不做“全学科”，只做函数与导数。

9.3 评测：怎么证明“说得准”

金标准：100 条真实/仿真对话，每条由两位教师独立标注：

- 最早断点节点；
- L 层；
- 是否可接受“多因”；
- 诊断卡是否给了可执行的下一步。

指标：

指标	目标	测量
卡点节点一致率	≥75%	系统 vs 双教师多数
L 层一致率	≥80%	同上
追问轮数	4–6	会话统计
首问延迟	<8s	P95
答案泄漏	0	红队 + 线上采样
“说中了”	≥70%	1–5 分，≥4 算中
24h 迁移	≥60%	新题同节点不复发
单次成本	≤¥0.8	LLM + OCR + 向量

离线评测集：

- 60 条“答案错”；
- 20 条“答案对但过程漏条件”；
- 10 条“完全不会”；
- 10 条“多卡点/超纲/情绪”负例。

9.4 成本

项	单次/单月	说明
LLM（4–6 轮）	¥0.3–0.6	小模型 + 32B 混合
OCR	¥0.05–0.15	按张
向量/数据库	¥0.05	摊销
单次合计	**≤¥0.8**	目标
1,000 次/月	≤¥800	含重试
基础设施	¥800–1,500/月	2C4G × 2 + 托管 DB

定价测试（不做复杂订阅）：

- 免费：3 次诊断；
- ¥9.9 / 10 次；
- ¥39 / 月（不限次，限公平使用）；
- 教师版 ¥99 / 月（班级卡点热力图 + 讲评脚本）。

先验证“有人愿意为一次准确诊断付费”，再谈年费。

9.5 风险登记

#	风险	概率	影响	触发信号	对策
R1	任意题覆盖不了	高	高	30% 题目无法建图	限定题型；超纲明确说“只能步骤提示”
R2	OCR 公式错	高	中	识别可用率 <80%	人工纠错入口；Mathpix 兜底；先文本后拍照
R3	诊断不准	中	高	一致率 <60%	缩题型；加人工复核；只给“可能卡点”
R4	答案泄漏	中	高	红队命中	四道闸；schema 无 solution；出口拦截
R5	追问太多，用户烦	中	中	第 3 轮流失 >30%	最多 6 轮；每轮可跳过；第 3 轮给阶段性卡
R6	用户只想要答案	高	中	70% 点“直接看答案”	保留看答案入口，但退出学习模式；用“先看一步”降低门槛
R7	内容标注太慢	高	高	W2 结束 <10 道深标	先做 10 道，跑通再扩；教研与工程同坐
R8	成本失控	中	中	单次 >¥1.2	路由；缓存；限制轮数；小模型
R9	未成年人隐私	低	高	原图留存	7 天删除；加密；家长同意；不做广告画像
R10	模型幻觉/错误解释	中	高	引用缺失	SymPy 校验；引用必须带 source；低置信不硬判

9.6 Kill criteria

出现任一，停止扩张，回到 10 道题重做：

- 50 次真实诊断后，卡点一致率 <60%；
- 用户“说中了” <50%；
- 70% 用户在第 2 轮就点“直接看答案”，且不愿回来；
- 单次成本 >¥2，且降不下来；
- 找不到 2 位老师愿意标 30 道题；
- 答案泄漏出现 1 次且无法复现修复。

Pivot 选项：

- 诊断不准 → 退化为“错因提示 + 人工复核”，不假装自动；
- 用户不愿对话 → 改为“先给 3 个可能卡点，让用户选”，再追问；

- 拍照太难 → 先做文本 + 教师端录题。

9.7 为什么是 8 周，不是 10 周

上一版方案想同时做“诊断 + 课程 + 题库 + 家长端 + 增长”，10 周也不够。这一版只做一件事：

> **给一道题 + 一个答案，能不能在 6 轮内说出一个让老师和学生都认的卡点。**

这件事跑通，再加题型、再加学科、再加商业化。跑不通，加再多功能都是包装。

9.8 诚实清单

现在没有什么	影响	计划
真实用户数据	所有指标是目标	8 周 beta 补
100 道题标注	还没开始	W1 先 10 道，W2 30 道
疗效证据	不能宣称提分	只做前后测，不承诺分数
全学科	只覆盖函数与导数	Phase 2
7 服务真实拆分	只有边界与配置	按触发条件演进
大规模 SEO	只有 100 页卡点库	Phase 2

10. 3 分钟演示与高频追问

10.1 演示脚本

时间	画面	台词
0:00 – 0:20	题目 + 学生答案	“这是一道导数题。学生答案是对的：a≥1/e。但他其实不会。”
0:20 – 0:40	系统只显示“先不看答案”	“我们没有给他讲整题，只问了一个更小的问题。”
0:40 – 1:20	第一轮：a=0 时 f’(x) 是什么	“他答 1/x，没有零点。系统再问：那 a=0 时 f(x) 会不会超过 0？”
1:20 – 1:50	第二轮：a=-1 反例	“他意识到 a<0 时函数一直增，会超过 0。系统没有纠正，是让他自己撞到反例。”
1:50 – 2:20	卡点卡出现	“卡点不是求导，是 L2 条件转译：用驻点前没确认 a>0。证据、排除项、置信度都在。”
2:20 – 2:45	3 分钟微干预 + 同构题	“只讲一个动作：先问驻点存不存在，再问是不是最大值。然后一道同构题。”
2:45 – 3:00	24h 迁移	“明天用新题验证，不用同一道。我们只看卡点是否复发。”

演示要点：

- 不要展示完整 24 题测评；这不是重点。
- 不要念技术栈；技术底牌放在追问里。
- 一定要展示“答案对但过程漏条件”的案例，这是与搜题/AI 家教最清晰的差异。
- 演示失败预案：LLM 超时 → 切预录视频；OCR 错 → 手动改符号；用户不回答 → 展示“降级二选一”。

10.2 高频追问

Q1：这不就是 ChatGPT 加个提示词?

> 不是。ChatGPT 的目标是“把题讲清楚”，我们的目标是“在 6 轮内定位一个认知动作”。这需要三样它没有的东西：题目的解法图、用户步骤的对齐、可更新的假设后验。提示词只是最外层；没有图、没有证据、没有停止条件，就会变成无限聊天。

Q2：凭什么说诊断准?

> 三层：① 30 道深标题由两位教研独立标节点与探针，一致率 ≥0.80；② 系统输出与双教师多数一致率 ≥75%；③ 用户“说中了” ≥70%。低于 0.55 置信度，我们只给“可能卡点”，不硬判。

Q3：用户只想要答案怎么办？

> 给入口，但退出学习模式。我们不做“道德绑架”；如果用户点“直接看答案”，系统给答案，但不记录诊断、不生成卡点卡。下一次他回来，仍然可以选“先诊断”。商业上，愿意为“看懂”付费的人，才是目标用户。

Q4：任意题都能诊断吗？

> MVP 不能。只做函数与导数的五类题型。超纲题明确说“当前版本只能做步骤级提示”，不假装能诊断。先在一个窄领域做到 75% 一致，再扩题型。

Q5：多 Agent 是不是过度设计？

> 如果让 4 个 Agent 自由聊天，是。我们用确定性状态机：Parser、Diagnostician、Coach、Verifier 各管一段，LLM 只在 Coach 里选“问什么”。这样可以单测、回放、降级。MVP 甚至可以先把 Diagnostician 和 Coach 合成一个进程，但契约不变。

Q6：为什么不做 12 分钟测评？

> 测评回答“你整体怎么样”，我们要回答“这道题你卡在哪”。前者需要常模、信效度、大量题；后者只需要一道题的过程。单题诊断更轻、更快、更贴近用户当下的问题。长期画像可以有，但不是入口。

Q7：商业模式是什么？

> 先按次：免费 3 次，¥9.9/10 次，¥39/月。再验证教师版：班级卡点热力图 + 讲评脚本，¥99/月。MVP 不追求规模，先验证“有人愿意为一次准确诊断付费”。

Q8：学生照片和作答数据怎么处理？

> 原图加密存 7 天，只留 OCR 文本；默认不展示原图；监护人同意；不做广告画像；一键删除。未成年人数据最小化，是我们设计的第一原则，不是合规补丁。

Q9：怎么防止 AI 直接给答案？

> 四道闸：检索阶段过滤 `solution`；Prompt/schema 没有 `solution` 字段；出口做 5-gram + SymPy 等价检测；100 条红队回归，泄漏率必须为 0。只靠“提示词说不要给答案”一定会破。

Q10：这个项目最大的风险是什么？

> 不是模型，是“内容标注”。30 道题 × 每道 7–10 个节点 × 探针，需要教研真的坐下来标。如果 W2 结束做不到 10 道深标，后面全是空中楼阁。所以我们的 kill criteria 第一条就是：找不到 2 位老师愿意标 30 道题，就停。

10.3 验收清单

演示前必须逐项通过：

- [ ] 3 道题端到端可跑：答案错 / 答案对但过程漏 / 完全不会。
- [ ] 每道题的首问 <8s。
- [ ] 每道题 ≤6 轮出卡点卡。
- [ ] 卡点卡有证据、排除项、置信度、下一练。
- [ ] 学习模式不出现最终答案。
- [ ] 用户点“我不同意”能进入复核队列。
- [ ] 24h 迁移题可生成、可记录。
- [ ] 断网/超时有降级页面。
- [ ] 手机端可拍照/粘贴。
- [ ] 数据删除可用。

11. 设计过程：5 个“无共享记忆”的子 Agent

> 这一章不是产品功能，而是说明这份方案是怎么被“吵”出来的。 5 个子 Agent 共享同一份事实包，但彼此看不到对方的输出；主 Agent 只做冲突裁决与整合。

11.1 为什么要无共享记忆

如果让 5 个 Agent 在同一个群里聊，会出现：

- 锚定：第一个说“做 12 分钟测评”的人，把后面的人带偏；
- 迎合：后面的 Agent 倾向同意前面的结论；
- 责任稀释：错了不知道是谁的假设。

无共享记忆 + 独立产出，强制每个角色从自己的专业视角把话说完，再由主 Agent 做“对抗性整合”。

11.2 五个角色与提示词

A 需求/对话设计师

```
你是学习产品的对话设计师。用户需求：学生发来一道不会/做错的题和自己的答案，
系统要通过追问定位他具体不会的点。请只输出：
1) 3 个真实用户场景；2) 一次 8 分钟对话的逐轮脚本；3) 5 条对话硬规则。
约束：不要设计题库/课程/测评；不要给完整解答；一次只问一个问题。
```

B 认知科学家/诊断引擎

```
你是认知科学与教育测量背景的算法设计师。请把“学生不会”拆成可观察、可追问、
可更新的假设空间，并给出：1) 卡点分类；2) 每类的信号与探针；3) 置信度更新与停止条件。
约束：不输出 IQ/诊断标签；必须能落到一道高中导数题。
```

C 后端/AI 架构师

```
你是 Java/Spring Cloud + RAG 架构师。3 人 8 周，要支持“单题诊断对话”。
请给出：1) 4 个部署单元与 7 个服务边界；2) 核心表；3) 一次对话的调用链；
4) 缓存/可观测性/降级；5) 如何体现 skill.txt 的 11 项能力。
约束：不要一上来 9 个微服务；不要为了用中间件而用。
```

D 教研/内容负责人

```
你是高中数学教研负责人。请给出：1) 100 道题如何选；2) 30 道深标怎么标；
3) 解法图节点与探针模板；4) 双人标注一致率怎么算；5) 哪些题必须放弃。
约束：不追求题量；必须能在一个月内标完 30 道。
```

E 红队/苛刻评审

> 你是投资人的技术顾问，任务是杀死这个项目。请攻击：
> 1）诊断是否伪科学；2）任意题覆盖；3）答案泄漏；4）用户是否愿意被追问；
> 5）3 人 8 周是否可能；6）成本与合规。
> 输出：按“致命/严重/可接受”分级，每条给一个验证实验。

11.3 主 Agent 裁决的 6 个冲突

#	冲突	A/B 方	裁决	理由
1	做 12 分钟测评，还是单题诊断	A 想测评；B 想单题	**单题诊断**	用户当下带来的是一道题；测评是另一个产品
2	对外给七维画像，还是 L0–L6 卡点	B 想七维；A 想人话	**L0–L6 主输出，七维只做探针镜头**	用户要“哪一步”，不要“能力分数”
3	4 个 Agent 还是 7 个	C 想 7 个；A 想 4 个	**MVP 4 个，目标 7 个**	3 人 8 周，先跑通状态机
4	4 个部署单元还是 9 个服务	C 想 9 个；D 想快	**4 单元，7 服务边界**	边界先立，进程后拆
5	学习模式要不要给答案	A 想锁；E 说用户会跑	**锁到诊断后，但给“直接看答案”出口**	不道德绑架；出口退出学习模式
6	1000 用户还是 40–60 beta	D 想规模；E 想验证	**40–60 beta**	先证明一致率，再谈增长

11.4 红队改变了什么

红队 E 的输出最短，但改变了三件事：

1. **删掉“12 分钟测评”**：它会把单题诊断做成另一个产品，且需要常模，8 周做不出来。
2. **把“七维画像”降级为内部探针**：对外只说卡点，避免“智商测试”联想。
3. **加 kill criteria**：如果 50 次诊断一致率 <60%，停止扩张，回到 10 道题。

11.5 这套方法怎么复用到别的项目

- 事实包只给一次，避免每个 Agent 重复读长文；
- 每个 Agent 的输出必须带“约束/取舍/验证实验”，不只给结论；
- 主 Agent 不做“平均”，做“冲突裁决”，并记录理由；
- 红队必须独立，且有权否定前四个；
- 最后用一份“诚实清单”收尾，写明没做什么。

11.6 与运行时多 Agent 的区别

	设计时子 Agent	运行时 Agent
目标	产出方案	完成一次诊断

	设计时子 Agent	运行时 Agent
记忆	无共享	共享 session state
编排	主 Agent 裁决	确定性状态机
输出	文档/表格	JSON/追问/卡点卡
失败	重跑/人工整合	降级/模板/人工复核

不要把“设计时多 Agent”当成“产品多 Agent”。 前者是方法，后者是架构；两者都要，但不能混为一谈。

附录

A. 诊断卡 JSON

```
{
  "session_id": "S_9001",
  "problem_id": "P_FUNC_001",
  "card": {
    "layer": "L2",
    "label": "条件转译 / 参数分类",
    "node_id": "n4",
    "one_line": "用驻点前，未确认 a>0 使 x=1/a 存在且落在 x>0",
    "evidence": [
      "在 a=0 时仍写 '令 f'(x)=0 得 x=1/a'",
      "在 a=-1 时未判断 f'(x) 是否有零点"
    ],
    "excluded": ["求导", "解不等式", "最终答案"],
    "confidence": 0.86,
    "confidence_label": "high",
    "next_action": {
      "type": "micro_lesson",
      "concept_id": "C_PARAM_EXIST",
      "duration_min": 3,
      "task_id": "T_SAME_001"
    },
    "verify": {
      "type": "transfer",
      "due_hours": 24,
      "item_id": "T_TRANS_001"
    },
    "disagree": false
  }
}
```

B. 追问生成 Prompt（节选）

```
【角色】
你是 SynapseMind 的 Socratic Coach。你不讲完整解法，只问一个能区分假设的问题。

【输入】
- 题目解法图：{{graph}}
- 学生步骤：{{steps}}
- 当前假设及后验：{{candidates}}
- 已问探针：{{asked_probes}}
- 轮次：{{turn}} / 6

【选择规则】
1. 只考虑后验最高的两个假设。
2. 选择能最大程度区分它们的探针。
3. 问题不超过 30 个汉字；必要时给 2-4 个选项。
4. 如果学生上轮说“不知道”，本轮降一级：二选一/具体数值/半成品。
5. 不出现最终答案、完整解法、关键结论。

【输出】
严格按 schema 输出 JSON。不要解释。
```

C. 解法图 JSON（示例）

```
{
  "problem_id": "P_FUNC_001",
```

```
  "stem": "f(x)=ln x - a x, a∈R; f(x)≤0 for all x>0; find a",
  "nodes": [
    {"id": "n1", "type": "READ", "text": "理解恒成立与定义域 x>0"},
    {"id": "n2", "type": "CONCEPT", "text": "转化为求 f 的最大值"},
    {"id": "n3", "type": "COMPUTE", "text": "f'(x)=1/x-a"},
    {"id": "n4", "type": "CONDITION", "text": "分 a≤0 / a>0；a>0 时 x=1/a 在定义域",
     "common_errors": ["E_NOCASE", "E_DOMAIN"]},
    {"id": "n5", "type": "CONCEPT", "text": "a>0 时 f''<0，x=1/a 为最大值"},
    {"id": "n6", "type": "COMPUTE", "text": "解 ln(1/a)-1≤0"},
    {"id": "n7", "type": "CHECK", "text": "验证 a=1/e 等号；a<1/e 反例"}
  ],
  "edges": [["n1","n2"],["n2","n3"],["n3","n4"],["n4","n5"],["n5","n6"],["n6","n7"]]
}
```

D. 术语表

术语	含义
卡点	最早让解题链断掉的认知动作，不是最后算错的地方
L0 - L6	读题/概念/条件转译/策略/执行/校验/资源状态
解法图	把题目拆成认知动作节点与依赖边
探针	用来区分假设的一个具体问题
后验	问完一个问题后，对某假设的置信度
微干预	3 分钟讲一个点 + 1 道同构题
迁移题	同结构、不同情境，用来验证卡点是否复发
答案单向阀	检索/schema/出口/回归四道闸，学习模式拿不到答案

E. 参考方向

1. Roediger & Karpicke (2006). Test-enhanced learning.
2. Cepeda et al. (2006). Distributed practice in verbal recall tasks.
3. Dunlosky et al. (2013). Improving students' learning with effective learning techniques.
4. Sweller (1988). Cognitive load during problem solving.
5. Chi et al. (1989). Self-explanations.
6. Bjork & Bjork (2011). Making things hard on yourself, but in a good way.
7. VanLehn (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems.
8. Koedinger et al. (2013). LearnLab / cognitive tutors.
9. Schneider & McGrew (2018). The Cattell-Horn-Carroll theory of cognitive abilities.
10. Plomin et al. (2016). Top 10 replicated findings from behavioral genetics.

> 说明：以上为方向性参考，不是“本项目已证明有效”的疗效证据。MVP 的效度指标是“与双教师标注一致率”和“24h 迁移”，不是分数提升。

F. 版本

- v0.1：偏题版本（12 分钟测评 + 学习 OS + 增长），已归档。
- v0.2：单题诊断引导版（当前）。核心变化：从“测整体能力”改为“定位一道题的具体卡点”；从“完整课程”改为“3 分钟微干预”；从“10 周大平台”改为“8 周窄 MVP”。